CODING EXERCISE

Requirements specification and acceptance criteria

Longest increasing run

This baseline turns the coding-exercise brief into checks a reviewer can mark pass or fail. It covers the function, the eleven published cases, the C# submission, and the secondary engineering objectives.

Document	Requirements specification and acceptance criteria
Identifier	SRS-LIS-001
Version	1.0
Date	17 October 2026
Status	Baseline for acceptance
Source	code-test.md, the coding-exercise brief
Subject	Longest strictly increasing run
Platform	C# on .NET

A solution is accepted when every Must criterion passes. Should criteria are scored and recorded. They do not overturn a solution that meets every Must criterion.

Contents

How to read the identifiers . . . 2
1. Purpose and authority . . . 3
1.1 Purpose . . . 3
1.2 What the brief asks for . . . 3
1.3 Authority . . . 3
1.4 Acceptance decision . . . 3
2. Normative interpretation . . . 3
2.1 Terms . . . 3
2.2 Decisions . . . 4
2.3 Worked example: AT-10 . . . 4
2.4 Worked example: AT-11 . . . 4
2.5 Worked example: AT-01 . . . 4
3. Functional requirements . . . 4
4. Solution and engineering requirements . . . 7
5. Repository and submission requirements . . . 8
6. Verification requirements . . . 8
7. Secondary objectives . . . 9
8. Published acceptance tests . . . 10
8.1 Input identity . . . 10
8.2 Short inputs in full . . . 10
8.3 Long inputs . . . 12
9. Rule checks . . . 12
10. Traceability . . . 13
11. Assumptions and exclusions . . . 15
11.1 Assumptions . . . 15
11.2 Exclusions . . . 15
11.3 Fixture file . . . 15

How to read the identifiers

Must items are required for acceptance. **Should** items are the brief’s secondary objectives. FR is a functional requirement, QR a quality requirement, SR a submission requirement, VR a verification requirement, and NR a secondary objective. AT is a published case from the brief. RC is a rule check that makes the general rule testable beyond those eleven strings. AC is an acceptance criterion. D is an interpretation decision.

1. Purpose and authority

1.1 Purpose

This document is the acceptance baseline for the coding exercise in **code-test.md**. A reviewer uses it to decide whether a submitted C# solution meets the brief. The author of a solution uses it to see which behaviour, tests, and repository conditions are required.

1.2 What the brief asks for

The primary objective is a working solution that shows C# and .NET, readable code, unit tests, and instructions a reviewer can follow. The problem is one function. It receives a string of integers separated by single spaces and returns the longest increasing subsequence. When more than one subsequence has that longest length, the function returns the earliest. Supporting functions are allowed. The function must pass the eleven cases in the brief.

The secondary objectives are GitHub Actions, Docker, linting, and coverage reporting. They are scored separately in Section 7.

The submission is a public GitHub repository whose name is a UUID version 4. Work is committed as it proceeds. The commissioning organisation’s name is left out of the code, the documentation, and the history. When the work is finished, the repository URL is sent to the person who set the exercise.

1.3 Authority

The published inputs and outputs are normative. Section 2 records three decisions that are required to apply those outputs consistently. Where a textbook reading of “subsequence” would allow skipped values, the published output of AT-10 is the authority.

This document does not name the organisation that issued the exercise.

1.4 Acceptance decision

Mark each Must criterion pass or fail. The solution is accepted only when every Must criterion passes. One failed published case is enough to reject the functional behaviour. Record each Should criterion as met or not met. A missing Dockerfile, workflow, linter, or coverage report leaves the primary decision unchanged.

2. Normative interpretation

2.1 Terms

Term	Meaning in this baseline
Input sequence	The integers produced by splitting the input on a single space and reading each token as a base-10 integer.
Run	A contiguous segment. Order is preserved and no integer between the ends of the segment is left out.
Strictly increasing	Each value after the first is greater than the value immediately before it. The difference may be any positive number.
Length	The number of integers in the run.
Earliest	Among runs of equal length, the run whose first integer has the smallest index.
Observable result	The space-separated base-10 string a test compares with the expected output. No leading or trailing whitespace.
Published case	One of AT-01 through AT-11, taken from the fenced examples in the brief, in that order.

2.2 Decisions

These three decisions are part of the baseline. They exist so that two reviewers, working from the brief alone, reach the same pass or fail.

Decision	Rule	Why it is fixed this way
D-01	The accepted sequence is a contiguous run.	AT-10 requires 2 4 6. Skipping values produces a longer selection and fails the published output.
D-02	Earliest means the smallest starting index.	The brief says to output the earliest sequence when several share the longest length. On AT-10, 2 4 6 starts before 1 5 9.
D-03	An increase is strict.	“Increased by any number” is a positive change of any size. An equal neighbour has not increased. RC-05 makes that rule testable, because the published suite has no adjacent equals.

2.3 Worked example: AT-10

Input 6 2 4 6 1 5 9 2. The contiguous strictly increasing runs include Run A, 2 4 6, and Run B, 1 5 9. Both have length 3. Run A starts earlier.

Index	0	1	2	3	4	5	6	7
Value	6	2	4	6	1	5	9	2
Run A		**2**	**4**	**6**				
Run B					**1**	**5**	**9**	

Run A is the required result. Run B is a longest run that starts later, so it is not returned.

A selection such as 2 4 5 9 skips the second 6 and is longer than Run A. It is not a run under D-01, and it is not the published output.

2.4 Worked example: AT-11

Input 6 2 4 3 1 5 9. The runs are 6, 2 4, 3, and 1 5 9. The longest run is 1 5 9. The earlier run 2 4 is shorter, so the tie-break does not apply. The required output is 1 5 9.

2.5 Worked example: AT-01

Input 6 1 5 9 2. The longest run is 1 5 9. The leading 6 is a run of length 1, and the final 2 does not extend 1 5 9.

3. Functional requirements

Each requirement states the behaviour and the criteria that demonstrate it. Examples written with a monospaced face are exact strings.

FR-01 **MUST**

Accept one string of space-separated integers

The solution provides one function that takes a single string. That string holds the integers of the problem, separated by a single space character (U+0020). Each published case is one line: no leading space, no trailing space, and no newline inside the string passed to the function.

The count of integers is not fixed. The largest published input, AT-09, contains 8,192 integers. Every published value is an integer from 1 to 32,762 inclusive, so a 32-bit signed integer is sufficient for the published suite. The function compares values numerically.

Supporting functions are allowed. The brief does not prescribe the function name, the project layout, or the algorithm.

Acceptance criteria

AC-01.1 The function can be called with one string argument. No second argument is required to select formatting, length, or tie-break behaviour.

AC-01.2 Each published input splits on a single space into the element count in Section 8, and every token parses as a base-10 integer.

AC-01.3 Calling the function with the 8,192-element input AT-09 returns the expected string for that case. The input is in scope for the automated test run.

FR-02 MUST

Return the longest strictly increasing run

A **run** is a contiguous segment of the input. Values stay in their original order, and no value lying between the first and the last chosen value is omitted.

A run is **strictly increasing** when each value after the first is greater than the value immediately before it. The result is a strictly increasing run whose length is greater than or equal to the length of every other strictly increasing run in the input.

Decision D-01 fixes this meaning for acceptance. The brief says “subsequence”. The published outputs are the longest contiguous runs. On AT-10 the contiguous result `2 4 6` is required. Skipping values to build `2 4 5 9` or `2 4 6 9` fails that case.

Acceptance criteria

AC-02.1 Every pair of neighbours in the result is strictly increasing, and the result occurs as a contiguous segment of the input.

AC-02.2 No strictly increasing run in the input is longer than the result.

AC-02.3 Given `6 1 5 9 2`, the result is `1 5 9`. This is AT-01.

AC-02.4 Given `6 2 4 6 1 5 9 2`, the result is `2 4 6`. The strings `2 4 5 9` and `2 4 6 9` fail AT-10.

FR-03 MUST

Treat any positive step as an increase

The brief says the sequence is “increased by any number”. For acceptance, each step is a positive difference of any size. Successive values do not have to differ by one.

Acceptance criteria

AC-03.1 Given `1 100 101`, the result is `1 100 101`. This is RC-06.

AC-03.2 AT-02, AT-03, and AT-09 return their expected runs. Those runs contain steps greater than 1.

FR-04 MUST

Break length ties by choosing the earliest run

When more than one strictly increasing run has the maximum length, the function returns the earliest. Earliest means the run whose first element has the smallest index in the input.

Decision D-02 records that reading. On AT-10 the runs `2 4 6` and `1 5 9` both have length 3. The required output is `2 4 6`, which starts at index 1. Indexes are zero-based in this explanation; the implementation does not have to expose indexes.

Acceptance criteria

AC-04.1 When two or more strictly increasing runs share the maximum length, the returned run has the smallest starting index.

AC-04.2 Given `1 3 2 4`, the result is `1 3`. This is RC-04. The later run `2 4` has the same length.

AC-04.3 Given `6 2 4 6 1 5 9 2`, `2 4 6` is returned and `1 5 9` is not. This is AT-10.

AC-04.4 Given `9 7 5 3`, the result is 9. This is RC-03. Each value is a run of length 1, and the first value starts earliest.

FR-05 MUST

Emit the run as a single space-separated string

The observable result is one string. It contains the chosen integers in order, written in ordinary base-10 form, separated by a single space. There is no leading whitespace, no trailing whitespace, and no trailing newline. A run of one integer is that integer alone.

The function may return that string directly, or it may return the integers in order. In the second form, the tests and the verification instructions show the rendering that produces this exact string. Published tokens have no leading zeros and no leading plus signs, and the expected strings match those tokens.

Acceptance criteria

AC-05.1 For every case AT-01 through AT-11, the observable string is identical to the expected string in Section 8.

AC-05.2 Given 42, the observable string is 42. This is RC-01.

AC-05.3 Given `1 2 3 4`, the observable string is `1 2 3 4`. This is RC-02.

AC-05.4 A result that matches the expected integers and then adds a trailing space or newline fails the exact-string comparison.

FR-06 MUST

End a run when the next value does not increase

An equal neighbour has not increased. The run ends when the next value is less than or equal to the current value. Decision D-03 records this reading of “increased”.

None of AT-01 through AT-11 contains adjacent equal values. A solution that treated equals as a continuation would still pass the published suite. RC-05 is therefore part of acceptance, so the comparison rule is fixed for every reviewer.

Acceptance criteria

AC-06.1 Given `1 2 2 3`, the result is `1 2`. This is RC-05. The later run `2 3` has the same length and a later start.

FR-07 MUST

Apply the same numeric rule to every integer

The brief asks for integers, and the published suite happens to use only positive values. Zero and negative integers are still integers. They are ordered by numeric value, with the same run and tie-break rules.

Acceptance criteria

AC-07.1 Given `-5 -1 0 4 1`, the result is `-5 -1 0 4`. This is RC-07. The final 1 is smaller than 4, so it ends the run.

AC-07.2 Given `1 2 3 0 1`, the result is `1 2 3`. This is RC-08.

FR-08 MUST

Pass the eleven published cases

The brief says the function should pass the test cases supplied with the exercise. Those cases are normative. Section 8 is the catalogue. The input identity of each case is the SHA-256 digest of its exact string. The machine-readable copy is `acceptance-fixtures.json`, embedded in this PDF and stored beside it.

A function that returns the eleven expected strings only for those exact inputs, and fails the rule checks in Section 9, does not meet FR-02 through FR-07.

Acceptance criteria

AC-08.1 For each case AT-01 through AT-11, the observable output equals the expected string in Section 8.

AC-08.2 The input used for each published case has the SHA-256 digest listed for that case. The digest is computed over the UTF-8 bytes of the string, with no byte-order mark and no trailing newline.

AC-08.3 The function also returns the expected string for RC-01 through RC-08.

4. Solution and engineering requirements

These requirements come from the primary objectives in the brief: C# and .NET, readable code, and unit tests.

`QR-01` **MUST**

Implement the solution in C# on .NET

The brief requires a C# .NET solution and a clear demonstration of that platform. The code that implements the function, and the tests that call it, are C#. A supported .NET SDK builds and tests the solution with the `dotnet` tool.

The brief does not name a target framework. The verification instructions name the SDK major version a reviewer needs.

Acceptance criteria

AC-09.1 The function under test is written in C#, in a project the .NET SDK can build.

AC-09.2 From a clean checkout, the documented `dotnet` commands restore, build, and test the solution.

AC-09.3 The verification instructions state the required .NET SDK version.

`QR-02` **MUST**

Keep the solution readable

The brief asks for clean code that is easy to read and understand. Acceptance uses observable checks rather than a style preference.

Acceptance criteria

AC-10.1 The verification instructions name the project, type, and method a reviewer should read first.

AC-10.2 The method name, or the names immediately around it, tell a reader that the method finds the longest increasing run.

AC-10.3 `dotnet build` completes with no compiler errors. Any compiler warning that remains is explained in the verification instructions.

AC-10.4 The eleven published results are produced by the run algorithm. A table that maps those eleven inputs to their outputs, and does not implement the rule, fails AC-08.3.

`QR-03` **MUST**

Unit-test the function

The brief asks the author to demonstrate unit testing. The automated suite is the evidence. Tests call the function directly. A manual console session is not a substitute for the suite.

Acceptance criteria

AC-11.1 The suite includes AT-01 through AT-11. Each test asserts exact equality with the expected string in Section 8.

AC-11.2 `dotnet test` exits successfully when those tests pass.

AC-11.3 The tests are deterministic. They do not depend on the network, the clock, or a particular machine name.

AC-11.4 A failing published case causes `dotnet test` to fail.

5. Repository and submission requirements

These requirements come from the “Get started” instructions in the brief.

SR-01 MUST

Publish a public GitHub repository named with a UUID

The brief asks for a public GitHub repository. The repository name is a UUID version 4, in the canonical hyphenated textual form recommended by the generator linked from the brief. Lower-case hexadecimal is the accepted form.

Acceptance criteria

AC-12.1 The repository is on GitHub and its visibility is public. The code is visible without signing in.

AC-12.2 The repository name matches this lowercase pattern, and nothing else:
`^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$`

SR-02 MUST

Commit while the solution is being developed

The brief asks the author to keep committing as the solution is developed, and then to share the repository URL with the person who set the exercise.

Acceptance criteria

AC-13.1 The default branch contains at least two commits that add or change C# source or test files. A history whose only solution commit is one finished snapshot does not meet this criterion.

AC-13.2 The repository URL has been given to the person who set the exercise. Acceptance of the submission is recorded by that person.

SR-03 MUST

Leave the commissioning organisation unnamed

The brief says not to commit the company name in any documentation or code. This baseline does not repeat that name. The check covers files in the repository and the commit messages on the submitted history.

Acceptance criteria

AC-14.1 A case-insensitive search of the repository files and of `git log` does not find the name of the organisation that issued the exercise.

6. Verification requirements

VR-01 MUST

Tell a reviewer how to verify the solution

The brief asks for clear instructions on how to verify the solution. A reviewer who has the required .NET SDK, and nothing else installed for this exercise, can follow those instructions from a clean checkout.

Docker, continuous integration, linting, and coverage are secondary. The primary instructions stand on their own when those extras are absent.

Acceptance criteria

AC-15.1 The instructions include the SDK version, the build command, the test command, and the location of the function under test.

AC-15.2 A reviewer who follows the instructions reaches a successful `dotnet test` run on a checkout of the submitted commit.

AC-15.3 The instructions are in the repository. They do not depend on a private document or an unstated local path.

7. Secondary objectives

The brief lists these as secondary objectives. Each one has acceptance criteria so it can be marked met or not met. None of them is required for the primary acceptance decision in Section 1.4.

`NR-01` SHOULD

Build and test continuously with GitHub Actions

A secondary objective is continuous integration with GitHub Actions. This criterion is scored. It does not overturn a solution that meets every Must criterion.

Acceptance criteria

AC-16.1 A workflow file under `.github/workflows` runs on push to the default branch.

AC-16.2 That workflow builds the solution and runs the unit tests, including AT-01 through AT-11.

AC-16.3 The workflow run for the submitted commit succeeds on GitHub.

`NR-02` SHOULD

Containerise the solution with Docker

A secondary objective is containerisation with Docker. The image builds from the repository using a documented command.

Acceptance criteria

AC-17.1 The repository contains a Dockerfile, and the verification instructions include the `docker build` command.

AC-17.2 Following those instructions produces an image. A documented command runs the unit tests in that image, or runs the function against a published case and shows the expected string.

`NR-03` SHOULD

Lint the code

A secondary objective is code linting. The tool is the author's choice. `dotnet format`, editorconfig rules, and Roslyn analyzers all qualify when the command is documented and it is actually run.

Acceptance criteria

AC-18.1 The repository configures a linter and the instructions include the command that runs it.

AC-18.2 Running that command on the submitted commit reports no remaining issues, or the workflow in NR-01 runs the same command and succeeds.

`NR-04` SHOULD

Report code coverage

A secondary objective is code coverage reporting. The brief sets no percentage threshold, and this baseline does not add one. The report has to be produced, and the published cases have to be part of the run being measured.

Acceptance criteria

AC-19.1 A documented command produces a coverage report for the unit-test run.

AC-19.2 The measured run includes AT-01 through AT-11. The instructions say where the report is written.

8. Published acceptance tests

The eleven cases below are the fenced examples in the brief, in the order they appear. AT-01 is the first example and AT-11 is the last. The expected string is exact.

Case	Input integers	Result length	Expected output
AT-01	5	3	1 5 9
AT-02	2,048	6	1710 2461 9288 10195 10431 12485
AT-03	4,096	8	10298 10897 12291 15037 18446 23435 25333 27266
AT-04	64	4	3862 16353 22813 28735
AT-05	128	4	11084 11970 24975 30922
AT-06	256	4	3808 3908 10386 19306
AT-07	512	6	125 1841 5882 18464 28317 31497
AT-08	1,024	6	9139 17687 25106 26202 27592 30937
AT-09	8,192	8	918 1089 5133 7725 18035 24605 26716 27095
AT-10	8	3	2 4 6
AT-11	7	3	1 5 9

Across the eleven inputs there are 16,340 integers. The smallest published value is 1 and the largest is 32,762. No published input contains a zero, a negative, or two equal neighbours.

8.1 Input identity

Compute SHA-256 over the UTF-8 bytes of the input string. Do not include a byte-order mark or a trailing newline. The digest must equal the value in this table. The same strings are stored in `acceptance-fixtures.json`.

Case	SHA-256 of the exact input string
AT-01	67749029879adb631d8e564186f1408bc48485314e268d0b2d1bda78dc514d49
AT-02	be8ffa8861f17b0b948ed570118e37c03621bdb7e6c22b5a644d2ede2d1000ee
AT-03	c677ac5b4c781db9323f0b6439e92106c509fc700c9dbf4acc11d0d6ce582600
AT-04	9cf7d4672aa22a2cca00df69288c607b94e712ee30b3d0d4579ea9642345b390
AT-05	d396b66609be2e1670b97e753837b972a1adb0d12d6041af5d257c8d07e386d8
AT-06	c3566bbb39399f2f5ec4ff98c5a70c785cd76f462eff670b7305bc0926ad7267
AT-07	34ba1a122adf089b1228e35c78a150353aadbda5f02a56532c0520838bc2075f
AT-08	3a8933f81d8eb58c13e13948af59f35104c8d80ac1d7e1513a4425df366b3da0
AT-09	99e67e347a9b06bc1dd8cc231745d9de1e54c9c6a6879a04ec5086574dc45612
AT-10	b75ae09ff7da3f931280674ab6142ffa6e01acc1eb094f57db19238b750d7f13
AT-11	b86051fa8be603368bbe950dd23f76fdf5e13795af07a883c2123f60d1c20b81

8.2 Short inputs in full

AT-01, AT-04, AT-05, AT-06, AT-10, and AT-11 are short enough to print in full. Copying from the fixture file is the reliable way to obtain the exact bytes.

AT-01 · 5 integers · result length 3

Input

```
6 1 5 9 2
```

Expected output

```
1 5 9
```

AT-04 · 64 integers · result length 4

Input

```
923 11613 30483 19569 24201 13461 1189 30793 8848 16914 16053 21700 22116 3852 20909 5231 31469 3862 16353
22813 28735 4421 3618 32303 9932 31892 7823 22547 28888 11143 11695 3339 2094 11023 9661 27440 7186 24750
15427 24502 31606 23515 3563 29553 12145 22184 11409 28824 6636 10658 21404 5578 27807 14073 13967 31310
3132 4321 7643 1951 13289 24375 17912 11304
```

Expected output

```
3862 16353 22813 28735
```

AT-05 · 128 integers · result length 4

Input

```
27892 18536 13491 11084 11970 24975 30922 11945 15113 27101 1974 31902 2623 21822 11720 30730 23635 27193
17527 19799 16794 30488 8953 28856 12300 25162 32016 20910 30896 6661 9255 26577 12629 10032 24221 31949
26243 26495 18785 22443 10673 13024 30655 11602 20408 28694 17785 31309 29576 23715 3866 10702 4378 3052
17543 11763 19622 24984 2519 27977 14869 2873 23140 10639 14521 15662 25122 17340 14140 14024 304 323
29654 20907 11693 13973 3267 8311 10189 31463 29941 24744 13356 18742 8454 17339 20578 12937 112 21395
5591 1399 5888 30234 16089 3816 19080 21547 491 22560 14549 10160 14176 1529 10720 13575 32041 15727 29256
29611 19692 12642 23040 10768 14422 15768 23365 206 16305 13058 19924 20738 30393 14656 21081 12785 27563
26982
```

Expected output

```
11084 11970 24975 30922
```

AT-06 · 256 integers · result length 4

Input

```
4650 2543 1184 1537 10037 9856 18201 29781 16440 8124 15835 23273 21808 2808 28925 2374 19 16546 9279 3323
19905 14701 20381 6116 6968 18094 1572 7084 21256 10758 16133 16017 7944 20546 13544 3431 25158 13183
20354 3808 3908 10386 19306 7178 11873 7079 5160 8729 10389 39 1167 12647 10849 30688 334 32185 13062 30048
8641 13252 9206 23969 23309 18416 25736 27884 24939 8795 28143 25335 13252 19489 21357 14929 1407 475 6852
28415 31383 20325 30958 6248 27120 6373 18189 23150 6635 13576 17317 29150 11064 31887 29705 24586 9707
29451 14867 31156 23762 27876 29208 23854 6918 22176 11547 5056 20417 354 23361 25162 11375 6321 31365
3370 26888 9595 7597 4960 8860 16986 24445 9456 2096 25877 5674 22664 18888 24938 23314 25699 32197 2315
15017 24173 28785 3302 8375 18344 4569 937 21697 26318 30171 14726 6852 13880 6783 11572 19498 26878 26763
24118 31990 19296 8785 20575 322 613 13516 26774 26347 28353 7915 1751 14219 14076 25389 378 5799 15569
14190 26040 4138 469 26369 8555 11026 16217 6910 31253 5226 13712 31965 29988 30328 30197 26436 5492 20535
11495 6430 4630 24840 11926 7917 27216 31908 19235 26209 31460 23381 11779 12329 30869 14950 19621 21513
29672 24791 10324 2861 18984 23638 26462 13501 10156 2667 2422 18609 32605 16883 15191 27879 7026 12495
3498 24690 11134 16861 4906 12598 29613 7975 10758 28594 330 9307 29830 21788 2181 21884 17650 4019 20340
11734 176 10577 19031 23351 16044 15423 19744 14684 9469 27214 30259
```

Expected output

```
3808 3908 10386 19306
```

AT-10 · 8 integers · result length 3

Input

```
6 2 4 6 1 5 9 2
```

Expected output

```
2 4 6
```

AT-11 · 7 integers · result length 3

Input

```
6 2 4 3 1 5 9
```

Expected output

```
1 5 9
```

8.3 Long inputs

The remaining published inputs are identified by digest, length, and a short preview. The full string is in the fixture file.

Case	Count	Preview	Expected output
AT-02	2,048	13499 30345 29165 5316 … 1580 25463 5265	1710 2461 9288 10195 10431 12485
AT-03	4,096	2311 11333 14555 1947 … 29182 7232 30067	10298 10897 12291 15037 18446 23435 25333 27266
AT-07	512	8618 18885 18979 13930 … 14703 19583 4359	125 1841 5882 18464 28317 31497
AT-08	1,024	11509 13451 983 15160 … 23491 25746 2976	9139 17687 25106 26202 27592 30937
AT-09	8,192	17567 24252 29110 27723 … 9635 31188 29497	918 1089 5133 7725 18035 24605 26716 27095

Preview shows the first four integers, then the last three. It is a reading aid. The digest is the identity check.

9. Rule checks

These checks are not printed in the brief. They are acceptance criteria for the general rule in Section 3. A solution that returns the eleven published strings and fails a rule check has memorised the sample list. That solution fails AC-08.3.

The submitted unit tests must include AT-01 through AT-11. They may also include these rule checks. Whether or not the author added them, the function must return the expected string when a reviewer calls it with the input below.

Check	What it fixes	Input	Expected
RC-01	A one-element input is already a complete run.	42	42
RC-02	An input that is strictly increasing throughout is returned unchanged.	1 2 3 4	1 2 3 4
RC-03	Every value is its own run of length 1, so the earliest run is the first value.	9 7 5 3	9
RC-04	Two runs share the maximum length. The one with the smaller starting index is returned.	1 3 2 4	1 3
RC-05	An equal neighbour is not an increase, so the run ends. The earliest longest run is returned.	1 2 2 3	1 2
RC-06	The step may be any positive amount. A step of 1 is not required.	1 100 101	1 100 101
RC-07	Comparison is numeric, so zero and negative integers follow the same rule.	-5 -1 0 4 1	-5 -1 0 4
RC-08	A later run is ignored when an earlier run is strictly longer.	1 2 3 0 1	1 2 3

RC-01 through RC-08 are included in `acceptance-fixtures.json` under `ruleChecks`.

10. Traceability

The register lists every acceptance criterion in this baseline. Use it as the review checklist. Priority follows the parent requirement.

Criterion	Priority	Requirement
AC-01.1	Must	Accept one string of space-separated integers
AC-01.2	Must	Accept one string of space-separated integers
AC-01.3	Must	Accept one string of space-separated integers
AC-02.1	Must	Return the longest strictly increasing run
AC-02.2	Must	Return the longest strictly increasing run
AC-02.3	Must	Return the longest strictly increasing run
AC-02.4	Must	Return the longest strictly increasing run
AC-03.1	Must	Treat any positive step as an increase
AC-03.2	Must	Treat any positive step as an increase
AC-04.1	Must	Break length ties by choosing the earliest run
AC-04.2	Must	Break length ties by choosing the earliest run
AC-04.3	Must	Break length ties by choosing the earliest run
AC-04.4	Must	Break length ties by choosing the earliest run
AC-05.1	Must	Emit the run as a single space-separated string
AC-05.2	Must	Emit the run as a single space-separated string
AC-05.3	Must	Emit the run as a single space-separated string
AC-05.4	Must	Emit the run as a single space-separated string
AC-06.1	Must	End a run when the next value does not increase
AC-07.1	Must	Apply the same numeric rule to every integer
AC-07.2	Must	Apply the same numeric rule to every integer

Criterion	Priority	Requirement
AC-08.1	Must	Pass the eleven published cases
AC-08.2	Must	Pass the eleven published cases
AC-08.3	Must	Pass the eleven published cases
AC-09.1	Must	Implement the solution in C# on .NET
AC-09.2	Must	Implement the solution in C# on .NET
AC-09.3	Must	Implement the solution in C# on .NET
AC-10.1	Must	Keep the solution readable
AC-10.2	Must	Keep the solution readable
AC-10.3	Must	Keep the solution readable
AC-10.4	Must	Keep the solution readable
AC-11.1	Must	Unit-test the function
AC-11.2	Must	Unit-test the function
AC-11.3	Must	Unit-test the function
AC-11.4	Must	Unit-test the function
AC-12.1	Must	Publish a public GitHub repository named with a UUID
AC-12.2	Must	Publish a public GitHub repository named with a UUID
AC-13.1	Must	Commit while the solution is being developed
AC-13.2	Must	Commit while the solution is being developed
AC-14.1	Must	Leave the commissioning organisation unnamed
AC-15.1	Must	Tell a reviewer how to verify the solution
AC-15.2	Must	Tell a reviewer how to verify the solution
AC-15.3	Must	Tell a reviewer how to verify the solution
AC-16.1	Should	Build and test continuously with GitHub Actions
AC-16.2	Should	Build and test continuously with GitHub Actions
AC-16.3	Should	Build and test continuously with GitHub Actions
AC-17.1	Should	Containerise the solution with Docker
AC-17.2	Should	Containerise the solution with Docker
AC-18.1	Should	Lint the code
AC-18.2	Should	Lint the code
AC-19.1	Should	Report code coverage
AC-19.2	Should	Report code coverage

Source in the brief	Where it is tested
Longest increasing subsequence, earliest on a tie	FR-02, FR-03, FR-04, D-01, D-02, AT-01 to AT-11
One string of integers separated by single whitespace	FR-01, FR-05, Section 8
Pass the supplied test cases	FR-08, AC-08.1, AC-11.1
Supporting functions are allowed	FR-01. No layout or algorithm is mandated.
C# .NET, readable code, unit tests, verification instructions	QR-01, QR-02, QR-03, VR-01

Source in the brief	Where it is tested
Public GitHub repository named with a UUID, incremental commits, share the URL	SR-01, SR-02
Do not commit the company name	SR-03. The name is intentionally absent from this baseline.
GitHub Actions, Docker, linting, coverage	NR-01 to NR-04. Secondary. Scored, not gating.

11. Assumptions and exclusions

11.1 Assumptions

• Well-formed published inputs use a single space between integers and contain no leading or trailing space.

• The observable comparison is an exact string match against the expected output, including the spaces between numbers.

• A 32-bit signed integer can hold every published value. The baseline does not require arbitrary-precision arithmetic.

• The entry point may be a method on a class, a static method, or a function-shaped member. The instructions identify it.

• Returning a string, or returning the integers together with a documented rendering, are both acceptable when the observable string matches.

• Rule checks RC-01 through RC-08 are in force even though the brief did not print them. They stop a lookup of the eleven samples from passing.

11.2 Exclusions

The brief does not define the following inputs. Behaviour on them is out of scope. A reviewer does not fail a solution for the way it handles them, and does not require a solution to handle them.

• The empty string, or a string of only whitespace.

• Leading or trailing spaces, repeated spaces, tabs, or a newline inside the argument.

• Tokens that are not base-10 integers, including leading zeros, leading plus signs, decimal points, and thousands separators.

• Integers outside the range of a 32-bit signed integer.

• A performance budget, a required algorithmic complexity, or a coverage percentage.

• A required function name, a required console interface, or a required project layout.

• Authentication, persistence, a user interface, or any behaviour other than the function and the submission conditions in this baseline.

11.3 Fixture file

`acceptance-fixtures.json` is embedded in this PDF and is also stored in the Downloads folder beside it. It contains every published input, every expected output, every digest, and the eight rule checks. A test project may load that file. The strings there are the same strings hashed in Section 8.

End of SRS-LIS-001, version 1.0. The eleven published outputs and the three interpretation decisions are the functional baseline. The repository conditions in Sections 4 to 6 are required. The objectives in Section 7 are secondary.